

Aula 3: Aprendendo a rede — Parte 1

Do perceptron à minimização de risco empírico • por que diferenciabilidade é tudo

Prof. Marcelo Keese Albertini

GBC073 — Inteligência Computacional • Universidade Federal de Uberlândia
2026



- ▶ **O problema de aprendizado:** rede = função paramétrica $f(x; W)$; aprender = estimar W de *amostras*

- ▶ **O problema de aprendizado:** rede = função paramétrica $f(x; W)$; aprender = estimar W de *amostras*
- ▶ **Histórico:** regra do perceptron — e por que ela **não** treina redes de múltiplas camadas

- ▶ **O problema de aprendizado:** rede = função paramétrica $f(x; W)$; aprender = estimar W de *amostras*
- ▶ **Histórico:** regra do perceptron — e por que ela **não** treina redes de múltiplas camadas
- ▶ **A virada moderna:** ativações e perdas **diferenciáveis** — a decisão que tornou o deep learning possível

- ▶ **O problema de aprendizado:** rede = função paramétrica $f(x; W)$; aprender = estimar W de *amostras*
- ▶ **Histórico:** regra do perceptron — e por que ela **não** treina redes de múltiplas camadas
- ▶ **A virada moderna:** ativações e perdas **diferenciáveis** — a decisão que tornou o deep learning possível
- ▶ **ERM:** o arcabouço de todo treinamento em PyTorch

- ▶ **O problema de aprendizado:** rede = função paramétrica $f(x; W)$; aprender = estimar W de amostras
- ▶ **Histórico:** regra do perceptron — e por que ela **não** treina redes de múltiplas camadas
- ▶ **A virada moderna:** ativações e perdas **diferenciáveis** — a decisão que tornou o deep learning possível
- ▶ **ERM:** o arcabouço de todo treinamento em PyTorch
- ▶ **Ponte para a Aula 4:** treinar = otimizar uma função diferenciável $\Rightarrow$ descida de gradiente

Intuição

Uma ideia atravessa a aula: só ajustamos bilhões de parâmetros porque cada pedaço da rede responde “se eu mexer um pouquinho aqui, o erro muda quanto?”.

PARTE 1

0 problema de aprendizado



Recapitulação: redes representam qualquer função

- ▶ Redes neurais: **aproximadores universais** — funções booleanas, fronteiras de classificação e funções contínuas em compactos
- ▶ Desde que a arquitetura tenha **capacidade suficiente** (neurônios/camadas em número adequado)
- ▶ Voz $\rightarrow$ transcrição, imagem $\rightarrow$ legenda, estado do jogo $\rightarrow$ jogada: todas funções — logo, todas modeláveis por uma rede

Cuidado

“Existe uma rede que representa g ” **não** diz *como encontrá-la*: aproximação universal é teorema de existência; aprendizado é *busca*.

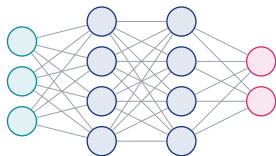
Intuição

Separar *representação* de *aprendizado*: a Aula 2 respondeu “o que a rede *pode* computar”; hoje, “como fazê-la computar o que queremos”.

Definição — Aprender a rede

Fixada a arquitetura, a rede computa $f(x; W)$, onde W reúne **todos os pesos e vieses**. Aprender é determinar W tal que $f(\cdot; W)$ aproxime a função-alvo g .

- ▶ A arquitetura (camadas, conexões): *decisão de projeto* — assumimos que ela basta
- ▶ Os **parâmetros** W : o que o treinamento ajusta
- ▶ Rede direta (*feedforward*): sem laços — realimentação fica para o futuro

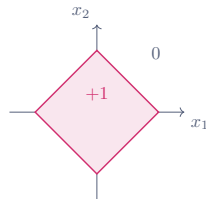


Cada seta carrega um peso w_{ji} ; o conjunto de todos eles é W .

- ▶ Fronteira simples (losango): dá para *projetar* os pesos — uma reta por lado, um E lógico no topo
- ▶ Reconhecer um dígito 28×28 ? Transcrever fala? **Impossível** escrever os pesos à mão
- ▶ Precisamos de **estimação automática**: calcular W que minimize a discordância entre $f(x; W)$ e $g(x)$

Intuição

Ideal: minimizar o desacordo $f \times g$ em *todo* o domínio — para isso seria preciso conhecer g em todo lugar; e aí não haveria o que aprender.



4 retas + 1 conjunção: fácil. Agora imagine 10^7 pesos.

Definição — Conjunto de treinamento

Não conhecemos g inteira; conhecemos **amostras** dela:

$$\mathcal{D} = \{(\mathbf{x}_1, d_1), (\mathbf{x}_2, d_2), \dots, (\mathbf{x}_N, d_N)\}, \quad d_i = g(\mathbf{x}_i).$$

Ex.: imagens e seus rótulos; áudios e suas transcrições.

- ▶ Ajustamos W para que $f(\mathbf{x}_i; W) \approx d_i$ *nos pontos amostrados*
- ▶ E **torcemos** para acertar onde não há amostra — a questão da *generalização*, tema recorrente do curso

Intuição

“Treinar” = “interpolar N pares entrada-saída com uma função paramétrica flexível, esperando que ela se comporte bem fora deles” — a frase descreve do perceptron ao GPT.

Para discutir em aula

Como redes neurais são aproximadores universais, *qualquer* rede, de *qualquer* arquitetura, aproxima qualquer função com precisão arbitrária — e, com poucas amostras de treinamento, seus parâmetros podem ser perfeitamente determinados.

Verdadeiro / Falso

Para discutir em aula

Como redes neurais são aproximadores universais, *qualquer* rede, de *qualquer* arquitetura, aproxima qualquer função com precisão arbitrária — e, com poucas amostras de treinamento, seus parâmetros podem ser perfeitamente determinados.

Verdadeiro / Falso

Fechamento

Falso, duas vezes. (1) A aproximação universal exige **capacidade mínima**: redes com poucos parâmetros podem ser aproximadores muito ruins. (2) Amostras finitas *subdeterminam* a função: infinitas redes passam pelos mesmos pontos. O que fazemos é *estimar* os parâmetros que melhor ajustam os dados — nunca “recuperar perfeitamente” a função-alvo desconhecida.

PARTE 2

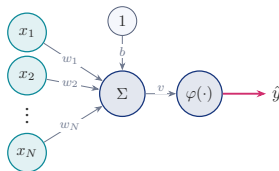
O perceptron de Rosenblatt (perspectiva histórica)



- ▶ Rosenblatt (1958): soma ponderada seguida de **degrau** —

$$\hat{y} = 1 \text{ se } w^\top x + b > 0; \hat{y} = 0 \text{ caso contrário}$$

- ▶ Truque de notação: o viés vira o peso w_{N+1} de uma entrada fixa $x_{N+1} = 1$
- ▶ Com isso a fronteira $w^\top x = 0$ é um hiperplano *pela origem* no espaço estendido



Definição — Geometria do perceptron

w é **ortogonal** ao hiperplano de decisão e aponta para a classe positiva. Aprender o perceptron = encontrar esse vetor normal.

Algoritmo — Regra de Rosenblatt (online)

Com rótulos $d_i \in \{-1, +1\}$:

- ▶ Inicialize $w = 0$
- ▶ Percorra as amostras; se $d_i (w^\top x_i) \leq 0$ (erro), atualize

$$w \leftarrow w + d_i x_i$$

- ▶ Repita até não haver erros

Errou um positivo? Some x_i (aproxima w dele). Errou um negativo? Subtraia. Só isso.

```
import numpy as np

def perceptron(X, y, epocas=100):
    # y em {-1, +1}; coluna do vies
    X = np.c_[X, np.ones(len(X))]
    w = np.zeros(X.shape[1])
    for _ in range(epocas):
        erros = 0
        for xi, yi in zip(X, y):
            if yi * (w @ xi) <= 0:
                w += yi * xi # Rosenblatt
                erros += 1
        if erros == 0:
            break
    return w
```

Teorema — Convergência do perceptron (Novikoff)

Se as classes são **linearmente separáveis**, o algoritmo converge em tempo finito: partindo de $w = 0$, faz no máximo $(R/\gamma)^2$ atualizações, onde $R = \max_i \|x_i\|$ e γ é a *margem* — a menor distância de uma amostra ao hiperplano separador ótimo.

- ▶ A mesma γ das SVMs: margens largas $\Rightarrow$ problema fácil
- ▶ Intuição do limitante: cada erro dá um passo de tamanho $\sim R$, e são precisos muitos passos de tamanho γ para desfazê-lo

Cuidado

Se as classes **não** são linearmente separáveis — o caso típico de dados reais, com ruído e sobreposição — não existe hiperplano perfeito e o algoritmo **nunca converge**: fica oscilando para sempre. Já aqui o modelo de 1958 esbarra na realidade.

Por que a regra do perceptron não treina um MLP

- ▶ Fronteiras não lineares exigem **camadas ocultas**: cada neurônio oculto aprende *uma* reta
- ▶ O dado só especifica **entrada e saída da rede** — ninguém diz o que cada neurônio oculto deve responder
- ▶ A regra exigiria *inventar rótulos intermediários* separáveis que, combinados, deem a fronteira certa — para *todos* os neurônios ao mesmo tempo



Cuidado

Rotular os neurônios internos é **busca combinatória** (exponencial). A constatação **paralisou a área por mais de uma década**: o MLP parecia intreinável.

Curiosidade histórica: ADALINE, MADALINE e a regra delta

Widrow & Hoff (1960) — o detalhe que sobreviveu foi a ideia de gradiente

- ▶ **ADALINE:** mesmo perceptron, outra regra de aprendizado — minimiza o erro quadrático sobre a *pré-ativação* v (contínua!), não sobre a saída binária
- ▶ **MADALINE:** versão multicamada com heurística gulosa (inverte a saída do neurônio “mais em dúvida”) — não escalou; nota de rodapé

$$E = \frac{1}{2} (d - v)^2 \quad \implies \quad \boxed{w \leftarrow w + \eta (d - v) x}$$

Intuição

A **regra delta** (LMS, Widrow–Hoff) é a descida de gradiente do erro quadrático em um neurônio linear. Generalizada para qualquer ativação diferenciável e encadeada camada a camada, ela vira a **retropropagação** — o algoritmo que treina toda rede moderna. De 1960 só descartamos o degrau; o gradiente ficou.

Para discutir em aula

Em um MLP cuja fronteira-alvo é um polígono, todos os neurônios já foram corretamente ajustados, exceto *um* neurônio oculto. Como os demais estão certos, podemos usar a regra do perceptron para aprender esse último neurônio diretamente dos dados de treinamento.

Verdadeiro / Falso

Para discutir em aula

Em um MLP cuja fronteira-alvo é um polígono, todos os neurônios já foram corretamente ajustados, exceto *um* neurônio oculto. Como os demais estão certos, podemos usar a regra do perceptron para aprender esse último neurônio diretamente dos dados de treinamento.

Verdadeiro / Falso

Fechamento

Falso. Os rótulos do conjunto de treinamento descrevem a saída da *rede*, não a saída desejada *daquele neurônio* — e, vistos por ele, os dados nem sequer são linearmente separáveis. Seria preciso *rerrotular* instâncias para criar o subproblema linear correto, e o dado não diz quais. É a versão em miniatura do problema combinatório que inviabiliza a regra do perceptron em redes profundas.

PARTE 3

Diferenciabilidade: a virada moderna



O verdadeiro vilão: derivada nula

- ▶ O erro de classificação é **binário**: deslocar levemente a fronteira quase nunca muda a contagem
- ▶ O degrau é **plano** (derivada 0) e não diferenciável no salto
- ▶ Mexer um pouquinho em um peso **não altera o erro** — não há sinal de *qual direção* melhora; o erro vira função em platôs: sem gradiente, sem bússola



Cuidado

O obstáculo nunca foi “multicamadas”; foi **a ausência de gradiente**. Com derivadas, o problema vira otimização contínua — tratável mesmo com bilhões de parâmetros.

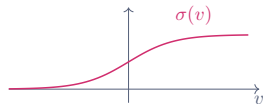
Definição — Requisitos de treinabilidade

(1) Ativações diferenciáveis — a saída de cada neurônio varia suavemente com pesos e entradas; **(2) perda diferenciável** — o desajuste rede $\times$ alvo varia suavemente com a saída da rede.

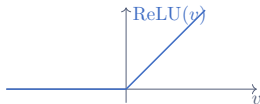
- ▶ Degrau $\rightarrow$ funções suaves: mudanças de peso produzem mudanças *mensuráveis* na saída
- ▶ “Conte os erros” $\rightarrow$ **divergência contínua**, um *proxy* do erro de classificação
- ▶ Pela **regra da cadeia**, a rede *inteira* vira diferenciável em cada parâmetro — o que a retropropagação explora (Aula 5)

Intuição

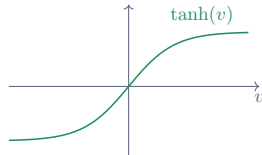
O coração do curso: *deep learning é a arte de manter tudo diferenciável*, para que o gradiente flua da perda a cada parâmetro.



Sigmoide $\sigma(v) = \frac{1}{1 + e^{-v}}$
saída em $(0, 1)$;
interpretação probabilística



ReLU $\max(0, v)$
padrão em redes profundas;
barata, gradiente 1 no lado
ativo



Tanh — sigmoide centrada
em 0; e primas modernas:
softplus, GELU, SiLU...

- Diferenciáveis *em quase todo lugar* basta: no “bico” da ReLU usamos uma **subderivada** (qualquer valor em $[0, 1]$) — e o treinamento nem percebe

A sigmoide é especial: nasce a regressão logística

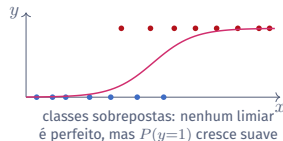
Definição — Regressão logística

Um perceptron com ativação sigmoide computa

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b),$$

a **probabilidade** da classe 1; decide-se $\hat{y} = 1$ se ela excede 0,5.

- ▶ Em dados reais as classes se *sobrepõem*: a fração de positivos cresce suavemente de 0 a 1 — o formato da sigmoide
- ▶ A saída vira **grau de confiança**, não um veredito



```
import torch

x = torch.tensor([1.0, 2.0])
w = torch.tensor([0.5, -0.3], requires_grad=True)    # parametros:
b = torch.tensor(0.1, requires_grad=True)            # pediremos gradiente

z = w @ x + b                                         # campo local induzido (diferenciavel)
y = torch.sigmoid(z)                                 # ativacao suave (diferenciavel)
d = torch.tensor(1.0)                                # alvo
loss = (y - d)**2                                     # divergencia (diferenciavel)
loss.backward()                                       # regra da cadeia, automatica
print(w.grad, b.grad)                               # dloss/dw e dloss/db
```

Intuição

Troque *sigmoid* por um degrau e *w.grad* vira zero em toda parte: o otimizador fica cego. Ativações suaves são a condição para o *backward()* existir.

Para discutir em aula

Com ativação degrau, a derivada do erro de classificação em relação ao limiar indica em que direção movê-lo. Já com ativação sigmoide, deslocar a função altera a distância total entre saídas e alvos mesmo quando o erro de classificação não muda.

Só a 1ª é verdadeira / Só a 2ª é verdadeira / Ambas / Nenhuma

Para discutir em aula

Com ativação degrau, a derivada do erro de classificação em relação ao limiar indica em que direção movê-lo. Já com ativação sigmoide, deslocar a função altera a distância total entre saídas e alvos mesmo quando o erro de classificação não muda.

Só a 1ª é verdadeira / Só a 2ª é verdadeira / Ambas / Nenhuma

Fechamento

Só a 2ª. Com degrau, o erro é constante por trechos: a derivada é 0 (ou inexistente) e *não* aponta direção alguma. Com sigmoide, a distância contínua aos alvos muda a cada deslocamento — e o seu gradiente diz para onde ir, *mesmo entre* duas configurações com a mesma contagem de erros. É exatamente por isso que otimizamos um **proxy contínuo**, e não o erro 0/1.

PARTE 4

Minimização de risco empírico



Definição — Função de divergência

$\text{div}(f(x; W), g(x))$ quantifica o desajuste entre a saída da rede e o alvo, satisfazendo:

- ▶ $\text{div}(f, g) = 0$ quando $f(x; W) = g(x)$;
 - ▶ $\text{div}(f, g) > 0$ quando diferem;
 - ▶ div é **diferenciável em relação a f** (e portanto, pela cadeia, em relação a W).
-
- ▶ Classificação: **não** é o erro 0/1 — é um substituto suave dele (veremos: entropia cruzada)
 - ▶ Regressão: o clássico $\text{div} = \|f(x; W) - g(x)\|^2$
 - ▶ A escolha da divergência: decisão de modelagem tão importante quanto a arquitetura

O que gostaríamos de minimizar — o **risco esperado** sobre todo o domínio de entrada:

$$R(W) = \mathbb{E}_{\mathbf{x}} [\text{div}(f(\mathbf{x}; W), g(\mathbf{x}))]$$

O que *conseguimos* calcular — a média sobre as N amostras:

$$\hat{R}(W) = \frac{1}{N} \sum_{i=1}^N \underbrace{\text{div}(f(\mathbf{x}_i; W), d_i)}_{\text{desajuste na amostra } i}$$

Definição — Minimização de risco empírico (ERM)

Treinar a rede é resolver $\hat{W} = \arg \min_W \hat{R}(W)$ — minimizar a *estimativa empírica* do risco, na esperança de que ela aproxime bem o risco verdadeiro.

Na prática chamamos $\hat{R}$ de *loss* (perda); fixado o conjunto de treinamento, a perda é função apenas de W .

Regressão: erro quadrático

$$\text{div} = \|\hat{\mathbf{y}}_i - \mathbf{y}_i\|^2$$

`nn.MSELoss` — suave, penaliza quadraticamente; ligado à verossimilhança gaussiana.

Classificação: entropia cruzada

$$\text{div} = - \sum_c d_{i,c} \log \hat{y}_{i,c}$$

`nn.CrossEntropyLoss` /
`nn.BCEWithLogitsLoss` — proxy
suave do erro 0/1; deriva da leitura
probabilística da sigmoide/softmax.

- ▶ As duas são diferenciáveis, convexas *na saída da rede* e têm gradientes bem-comportados — por isso dominam a prática
- ▶ Deduziremos a entropia cruzada com calma quando tratarmos de saídas probabilísticas; hoje basta o papel: **divergência no ERM**

ERM em PyTorch: o esqueleto de todo treinamento

```
import torch, torch.nn as nn

model = nn.Sequential(                #  $f(\cdot; W)$ : MLP diferenciável
    nn.Linear(2, 16), nn.ReLU(),
    nn.Linear(16, 1))
criterio = nn.BCEWithLogitsLoss()    #  $\text{div}()$ : proxy suave do erro 0/1
otim = torch.optim.SGD(model.parameters(), lr=0.1)
for epoca in range(1000):
    saida = model(X).squeeze()        #  $f(X_i; W)$  para todo  $i$ 
    risco = criterio(saida, d)         #  $\hat{R}(W)$ : média das divergências
    otim.zero_grad()
    risco.backward()                  # gradientes de  $\hat{R}$  em cada peso
    otim.step()                      # ajusta  $W$  na direção que reduz  $\hat{R}$ 
```

Intuição

Mapeie o laço na teoria: $model = f(\cdot; W)$, $criterio = \text{div}$, $risco = \hat{R}(W)$, $backward$ = regra da cadeia, $step$ = otimização. Do MLP aos LLMs, é este o programa.

- ▶ Enunciado puramente matemático: dado $\mathcal{D}$, minimizar $\hat{R}(W)$ em relação a W — **minimização de funções**
- ▶ Tudo diferenciável $\Rightarrow$ o gradiente $\frac{\partial \hat{R}}{\partial W}$ existe e aponta a direção de *maior crescimento* do risco — andaremos no sentido oposto

$$W \leftarrow W - \underbrace{\eta}_{\text{taxa de aprendizado}} \cdot \underbrace{\frac{\partial \hat{R}}{\partial W}}_{\text{gradiente do risco empírico}}$$

- ▶ **Aula 4:** descida de gradiente em detalhe — o que é o gradiente em muitas dimensões, mínimos locais, taxa de aprendizado
- ▶ **Aula 5:** como calcular esse gradiente *eficientemente* em uma rede profunda — a retropropagação

Exercício 1 — Da regra delta ao gradiente

Considere um neurônio linear $\hat{y} = \mathbf{w}^\top \mathbf{x}$ com perda $E = \frac{1}{2}(d - \hat{y})^2$ para uma única amostra $(\mathbf{x}, d)$.

- (a) Calcule $\frac{\partial E}{\partial \mathbf{w}}$ e mostre que a atualização por gradiente, com passo η , é exatamente a regra delta de Widrow–Hoff vista hoje.
- (b) Repita com $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x})$, usando $\sigma'(v) = \sigma(v)(1 - \sigma(v))$. Que fator novo aparece na atualização?
- (c) Verifique (b) numericamente com o trecho de *autograd* do slide anterior, comparando *w.grad* com sua fórmula.

- ▶ Aprender a rede = estimar os parâmetros W de $f(x; W)$ a partir de **amostras** (x_i, d_i) da função-alvo
- ▶ A regra do perceptron aprende *um* separador linear em tempo finito (classes separáveis) — mas em redes multicamadas exige rótulos intermediários desconhecidos: **busca combinatória**; ADALINE/MADALINE ficaram como curiosidade histórica
- ▶ O desbloqueio moderno foi tornar tudo **diferenciável**: ativações suaves (sigmoide, ReLU, ...) + divergência contínua como proxy do erro — o *autograd* é a materialização disso
- ▶ **ERM**: minimizar o risco empírico $\hat{R}(W) = \frac{1}{N} \sum_i \text{div}(f(x_i; W), d_i)$ — o laço de treinamento do PyTorch é ERM ao pé da letra
- ▶ Treinar virou **otimização de função diferenciável**; próxima aula: descida de gradiente